

Geo-Spatial Provider Search

Roger Arias, Irene Ramirez, Steven Weber, and
Michael McCarthy
University of North Florida, School of Computing
Jacksonville, Florida



INTRODUCTION

The Problem

- Static medical directories lack geographic intelligence.
- Finding specialists by name is insufficient without proximity data.

Motivation

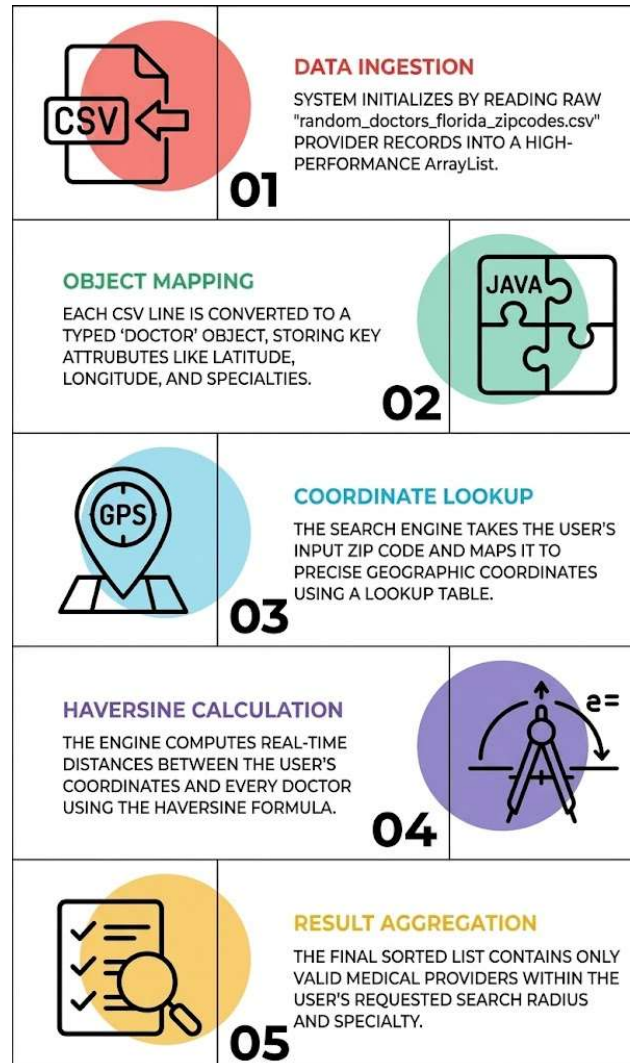
- Goal: Eliminate "healthcare deserts" by providing instant proximity results.
- Target: Patients seeking local specialized care and administrators managing provider networks.

The Solution

- In-Memory Search: Parses 10,000+ CSV records into Java objects for sub-second performance.
- Haversine Logic: Implements "Great-Circle" distance math to account for Earth's curvature.
- Workflow: Managed via professional Git integration and v1.0 tagging for stability.

OBJECTIVES

- **Standardized Data Integration:** Develop a robust parsing system to import and clean 10,000+ provider records from a specialized CSV dataset.
- **Geospatial Query Engine:** Implement the Haversine Formula to calculate precise distances between user-provided ZIP codes and medical facilities.
- **Multi-Criteria Filtering:** Enable users to narrow search results simultaneously by provider name, medical specialty, and geographic radius.
- **Scalable Architecture:** Design an in-memory data model that ensures sub-second search response times for large-scale datasets.
- **Professional Version Control:** Utilize a multi-branch Git workflow and annotated tagging to ensure code stability and seamless team integration.



This diagram illustrates the transition from raw CSV ingestion to the execution of the Haversine Formula. The modular design ensures that data parsing is decoupled from the search engine, allowing for sub-second query performance across 10,000+ records.

PROCEDURES

- **In-Memory Data Loading:** The system parses 10,000+ CSV provider records into a centralized ArrayList of Doctor objects during the application's startup phase.
- **Coordinate Mapping:** Upon receiving a user ZIP code, the engine performs a lookup to retrieve corresponding Latitude and Longitude coordinates.
- **Geospatial Calculation:** The system iterates through the provider list, applying the Haversine Formula to determine the "Great-Circle" distance between the user and each medical facility.
- **Automated Verification:** Unit tests, such as testSearch.java, are executed to validate parsing accuracy and ensure data integrity across different development branches.
- **Integrated Deployment:** Code contributions from the team are merged and verified through a structured Git workflow, culminating in a stable v1.0 release tag.

CONCLUSIONS

- **System Performance:** The in-memory architecture successfully processes 10,000+ provider records, delivering search results in sub-second timeframes.
- **Geospatial Accuracy:** By implementing the Haversine Formula, the application provides reliable "Great-Circle" distance filtering for Florida healthcare regions.
- **Structural Integrity:** A rigorous Git branching strategy and v1.0 tagging ensured a stable, integrated codebase across all contributor modules.
- **Verified Reliability:** Automated testing via JUnit confirmed that data parsing and object mapping remain consistent even as the dataset scales.